

CISTR

Práctica 2. Creación de tareas.
Uso de colas de mensaje y semáforos

José Manuel González Cava
jgonzalc@ull.edu.es

Objetivos

Conocer las funcionalidades de FreeRTOS para la creación de tareas y programación concurrente con semáforos y colas de mensaje.

- **Bloque 1** Creación de tareas.
- **Bloque 2** Colas de mensaje.
- **Bloque 3** Semáforos.

¿Qué es freeRTOS?

Sistema operativo open source en tiempo real. Puede usarse como SO para ESP-IDF (pero no es exclusivo).

- FreeRTOS se crea originalmente para trabajar con un único core (<https://www.freertos.org/index.html>).
- ESP-IDF ha desarrollado su propia versión (ESP-IDF FreeRTOS) para dispositivos multicore (<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html>).
- ESP-IDF FreeRTOS está basado en Vanilla FreeRTOS v10.4.3 al que se incorporan funcionalidades **SMP** para multicore

¿Qué es SMP?

SMP (Symmetric Multiprocessing) es una estructura donde dos o más CPUs idénticos se conectan a una memoria principal compartida y son controlados por un único SO.

Para el ESP32:

- Cuenta con dos cores idénticos (CPU0 y CPU1). La ejecución del código es independiente del core empleado.
- Tiene memoria simétrica compartida.
- Se permite la interrupción cruzada entre cores, permitiendo el envío de señales entre ellos.

Características de ESP-IDF FreeRTOS

- El planificado de tareas se inicia por defecto (vs. FreeRTOS)
- Se debe definir una función void app_main(void) como punto de entrada a la aplicación de usuario que se ejecuta al arrancar.
- FreeRTOS ESP-IDF crea varias tareas en segundo plano:
 - **Idle Task** para cada CPU. Tarea con prioridad baja para asegurar que hay, al menos, una tarea capaz de ejecutarse. No tiene función activa.
 - **Free RTOS Time Task**: Servicio de timers para FreeRTOS timer API.
 - **Main Task**: tarea que llama a app_main.
 - **IPC tasks**: tarea que permite la llamada entre CPUs.
 - **ESP Timer Task**. Servicio de timers propio de ESP.

Creación de una tarea

```
BaseType_t xTaskCreatePinnedToCore(TaskFunction_t pxTaskCode, const char
*const pcName, const configSTACK_DEPTH_TYPE usStackDepth, void *const
pvParameters, UBaseType_t uxPriority, TaskHandle_t *const pvCreatedTask, const
BaseType_t xCoreID)
```

Parameters:

- **pxTaskCode** – Pointer to the task entry function. Tasks must be implemented to never return (i.e. continuous loop), or should be terminated using vTaskDelete function.
- **pcName** – A descriptive name for the task. This is mainly used to facilitate debugging. Max length defined by configMAX_TASK_NAME_LEN - default is 16.
- **usStackDepth** – The size of the task stack specified as the number of bytes. Note that this differs from vanilla FreeRTOS.
- **pvParameters** – Pointer that will be used as the parameter for the task being created.
- **uxPriority** – The priority at which the task should run. Systems that include MPU support can optionally create tasks in a privileged (system) mode by setting bit portPRIVILEGE_BIT of the priority parameter. For example, to create a privileged task at priority 2 the uxPriority parameter should be set to (2 | portPRIVILEGE_BIT).
- **pvCreatedTask** – [out] Used to pass back a handle by which the created task can be referenced.
- **xCoreID** – If the value is tskNO_AFFINITY, the created task is not pinned to any CPU, and the scheduler can run it on any core available. Values 0 or 1 indicate the index number of the CPU which the task should be pinned to. Specifying values larger than (configNUM_CORES - 1) will cause the function to fail.

Returns:

pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

Creación de una tarea

- Las tareas sólo pueden estar en uno de los siguientes estados: Running, Ready, Blocked o Suspended.
- Las funciones de cada tarea se suelen implementar como un bucle infinito.
- Las funciones de tarea nunca deberían incluir un “return”.
- Una tarea puede eliminarse con `vTaskDelete()`. Se debe evitar eliminar tareas asignadas a otro core. Se prioriza el autoeliminación de una tarea.

Planificador en SMP

El planificador de FreeRTOS (Vanilla) es un **Fixed Priority Preemptive Scheduler (FPS)** con **Time Slicing**:

- **“Fixed priority”**: a cada tarea se le asigna una prioridad constante en su creación. El planificador ejecuta la tarea “Ready” con mayor prioridad.
- **“Preemptive”**: el planificador podrá parar una tarea de menor prioridad en ejecución para comenzar una tarea de mayor prioridad. La tarea de menor prioridad es “preempted”.
- **“Time Slicing”**: el planificador alternará de forma periódica la ejecución de tareas “Ready” que tengan la misma prioridad. El tiempo de cambio estará gobernado por un tick interrupt (<https://www.freertos.org/implementation/a00011.html>).

Planificador en SMP

ESP-IDF FreeRTOS introduce algunas diferencias:

- **Fixed priority:** cada Core planifica de forma independiente la tarea a ejecutar. Una tarea se ejecuta si:
 - i. La tarea es afín (asignada a ese core o a ninguno).
 - ii. La tarea no se está ejecutando por ningún otro core.

Como norma general, no se puede asumir que las dos tareas de mayor prioridad son ejecutadas en ese orden por el planificador.

- Task A of priority 10 pinned to CPU0
- Task B of priority 9 pinned to CPU0
- Task C of priority 8 pinned to CPU1

The resulting schedule will have Task A running on CPU0 and Task C running on CPU1. Task B is not run even though it is the second highest priority task.

Planificador en SMP

- **Preemption:** en caso de que una tarea en estado Ready pueda ejecutarse en varios core, el planificador elegirá uno. Por norma general, prioriza el core “actual” cuando varios pueden ser interrumpidos.

- Task A of priority 8 currently running on CPU0
- Task B of priority 9 currently running on CPU1
- Task C of priority 10 that is unpinned and was unblocked by Task B

The resulting schedule will have Task A running on CPU0 and Task C preempting Task B given that the scheduler always gives preference to the current core.

Planificador en SMP

- **Time slicing:** originalmente se plantea el algoritmo de planificación Round-Robin, en el que todas las tareas con la misma prioridad toman un turno para entrar al estado de ejecución.

Al haber más de un core, no se puede asumir que tareas con la misma prioridad se ejecuten de forma secuencial.

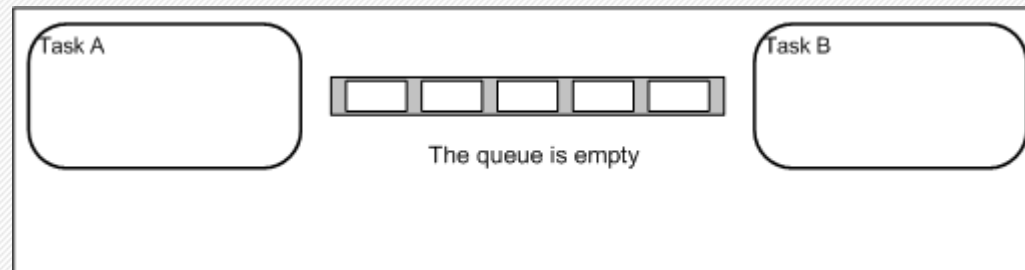
Se podría solucionar si las tareas de la misma prioridad se asignan al mismo Core.

Planificador en SMP

BLOQUE 1: Creación de tareas

FreeRTOS Queues

Permiten la comunicación entre tareas. En la mayoría de casos, se implementan como buffers FIFO:



<https://www.freertos.org/Embedded-RTOS-Queues.html>

```
xQueueCreate(uxQueueLength, uxItemSize)
```

Parameters:

- **uxQueueLength** – The maximum number of items that the queue can contain.
- **uxItemSize** – The number of bytes each item in the queue will require. Items are queued by copy, not by reference, so this is the number of bytes that will be copied for each posted item. Each item on the queue must be the same size.

Returns:

If the queue is successfully created then a handle to the newly created queue is returned. If the queue cannot be created then 0 is returned.

FreeRTOS Queues

- Cuando una tarea intenta leer una cola vacía, ésta se queda **bloqueada** hasta que:
 - i. Se reciba un elemento.
 - ii. Pase el tiempo de bloqueo.
- De la misma forma, una tarea que intenta escribir una cola llena se queda **bloqueada** hasta que:
 - i. Hay un hueco en la cola.
 - ii. Expira el tiempo de bloqueo.

Si varias tareas se quedan bloqueadas (lectura/escritura) se desbloqueará primero la de mayor prioridad.

Planificador en SMP

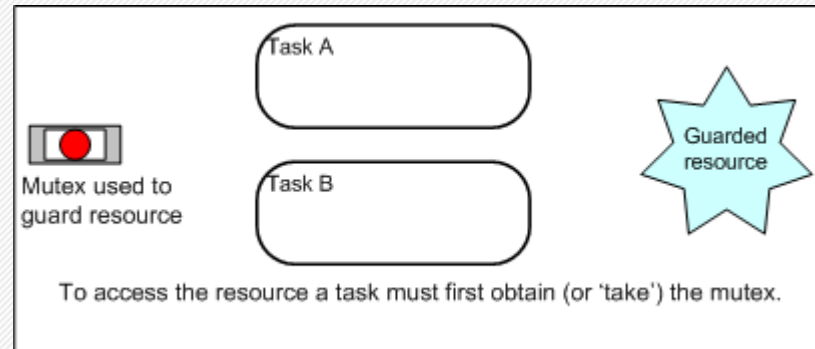
BLOQUE 2: Cola de mensajes

FreeRTOS Semaphores

Mutexes

Se emplean para gestionar una exclusión mutua.

El mutex actúa como un token que permite guardar un recurso. Para acceder a él, la tarea debe obtener el token (Take). Una vez termina, libera el recurso (Give)



FreeRTOS Semaphores

Semáforos binarios

Son muy similares a los Mutex, y son empleados cuando se debe gestionar una interrupción (sincronización).

La interrupción sólo hace un “Give” mientras que la tarea asociada a la activación hace un “Take”.



<https://www.freertos.org/Embedded-RTOS-Binary-Semaphores.html>

FreeRTOS Semaphores

Semáforos contadores

Se usan para:

- i. Contar eventos. El valor de conteo es la diferencia entre el número de veces que un evento ha ocurrido y el número de veces que ha sido procesado. Se inicializa a 0.
- ii. Gestión de recursos. El valor del contador indica el número de recursos disponibles. Se inicializa con el máximo valor de recursos disponibles.

FreeRTOS Semaphores

`xSemaphoreCreateMutex()`

Returns: If the mutex was successfully created then a handle to the created semaphore is returned. If there was not enough heap to allocate the mutex data structures then NULL is returned.

`xSemaphoreCreateBinary()`

Returns: Handle to the created semaphore, or NULL if the memory required to hold the semaphore's data structures could not be allocated.

`xSemaphoreCreateCounting(uxMaxCount, uxInitialCount)`

- **uxMaxCount** – The maximum count value that can be reached. When the semaphore reaches this value it can no longer be 'given'.
- **uxInitialCount** – The count value assigned to the semaphore when it is created.

Returns: Handle to the created semaphore. Null if the semaphore could not be created.

Planificador en SMP

BLOQUE 3: Semáforos

CISTR

Práctica 2. Creación de tareas.
Uso de colas de mensaje y semáforos

José Manuel González Cava
jgonzalc@ull.edu.es